

```

# =====
# SISTEMA INDUSTRIAL DE DIAGNÓSTICO ELÉTRICO INTELIGENTE
# VERSÃO ULTRA AVANÇADA
# =====

# !pip install -q pandas numpy matplotlib seaborn openpyxl

import json
import random
import time
import os
from datetime import datetime
from dataclasses import dataclass, asdict

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("darkgrid")

# =====
# CONFIGURAÇÕES GLOBAIS
# =====

MAX_HISTORICO = 500
ARQUIVO_LOG = "eventos.log"

# =====
# LOGGER
# =====

class Logger:

    @staticmethod
    def log(msg):
        timestamp = datetime.now().strftime("%H:%M:%S")

        linha = f"[{timestamp}] {msg}"

        print(linha)

        with open(ARQUIVO_LOG, "a", encoding="utf-8") as f:
            f.write(linha + "\n")

# =====
# MODELOS
# =====

```

```
@dataclass
class LeituraSensor:
    timestamp: str
    barramento: str

    tensao_Va: float
    tensao_Vb: float
    tensao_Vc: float

    corrente_la: float
    corrente_lb: float
    corrente_lc: float

    fator_potencia: float
    temperatura_C: float
    frequencia_Hz: float

    tensao_nominal: float
    corrente_nominal: float
```

```
@dataclass
class DiagnosticoFalha:
    barramento: str
    timestamp: str

    tipo_falha: str
    severidade: str
    severidade_score: int

    risco_futuro: str
    indice_saude: float

    causa_provavel: str

    recomendacoes: list

    laudo_tecnico: str
```

```
# =====
# SIMULADOR
# =====
```

```
class SimuladorRede:

    def __init__(self):

        self.Vn = 13800
```

```

self.ln = 400

def ruido(self, intensidade=0.02):
    return random.gauss(0, intensidade)

def gerar(self, barramento, cenario):

    Vn = self.Vn
    ln = self.ln

    if cenario == "normal":

        fator = 1

        tensao = [Vn * (1 + self.ruido(0.01)) for _ in range(3)]

        corrente = [ln * 0.75 for _ in range(3)]

        fp = 0.98
        temp = 42

    elif cenario == "sobrecarga":

        tensao = [Vn * 0.91 for _ in range(3)]

        corrente = [
            ln * 1.45,
            ln * 1.52,
            ln * 1.48
        ]

        fp = 0.86
        temp = 88

    elif cenario == "curto":

        tensao = [
            Vn * 0.08,
            Vn,
            Vn
        ]

        corrente = [
            ln * 9,
            ln * 0.7,
            ln * 0.68
        ]

```

```
fp = 0.32  
temp = 120
```

```
elif cenario == "desequilibrio":
```

```
tensao = [  
    Vn,  
    Vn * 0.84,  
    Vn * 1.15  
]
```

```
corrente = [  
    In * 0.7,  
    In * 1.25,  
    In * 0.62  
]
```

```
fp = 0.89  
temp = 72
```

```
elif cenario == "sobretensao":
```

```
tensao = [  
    Vn * 1.25,  
    Vn * 1.24,  
    Vn * 1.26  
]
```

```
corrente = [  
    In * 0.8,  
    In * 0.82,  
    In * 0.81  
]
```

```
fp = 0.94  
temp = 60
```

```
elif cenario == "baixo_fp":
```

```
tensao = [  
    Vn,  
    Vn,  
    Vn  
]
```

```
corrente = [  
    In * 0.96,  
    In * 0.95,
```

```

        ln * 0.94
    ]

    fp = 0.68
    temp = 65

    return LeituraSensor(
        timestamp=datetime.now().isoformat(),
        barramento=barramento,

        tensao_Va=round(tensao[0], 1),
        tensao_Vb=round(tensao[1], 1),
        tensao_Vc=round(tensao[2], 1),

        corrente_la=round(corrente[0], 2),
        corrente_lb=round(corrente[1], 2),
        corrente_lc=round(corrente[2], 2),

        fator_potencia=fp,
        temperatura_C=temp,
        frequencia_Hz=60.0,

        tensao_nominal=Vn,
        corrente_nominal=ln
    )

# =====
# IA DE DIAGNÓSTICO
# =====

class IAEletrica:

    def calcular_saude(self, leitura):

        score = 100

        corrente_max = max(
            leitura.corrente_la,
            leitura.corrente_lb,
            leitura.corrente_lc
        )

        corrente_pct = corrente_max / leitura.corrente_nominal

        if corrente_pct > 1:
            score -= corrente_pct * 30

        if leitura.fator_potencia < 0.92:

```

```

        score -= (0.92 - leitura.fator_potencia) * 100

    if leitura.temperatura_C > 70:
        score -= (leitura.temperatura_C - 70)

    return max(0, round(score, 2))

def detectar_anomalia(self, leitura):

    valores = np.array([
        leitura.tensao_Va,
        leitura.tensao_Vb,
        leitura.tensao_Vc,
        leitura.corrente_Ia,
        leitura.corrente_Ib,
        leitura.corrente_Ic
    ])

    z = np.abs((valores - valores.mean()) / valores.std())

    return np.max(z) > 2.5

def prever_risco(self, leitura):

    if leitura.temperatura_C > 90:
        return "ALTO"

    if leitura.fator_potencia < 0.8:
        return "MÉDIO"

    return "BAIXO"

# =====
# MOTOR PRINCIPAL
# =====

class MotorDiagnostico:

    def __init__(self):

        self.ia = IAEletrica()

    def diagnosticar(self, leitura):

        corrente_max = max(
            leitura.corrente_Ia,
            leitura.corrente_Ib,
            leitura.corrente_Ic

```

```

)

corrente_pct = (
    corrente_max / leitura.corrente_nominal
) * 100

indice_saude = self.ia.calcular_saude(leitura)

risco = self.ia.prever_risco(leitura)

if corrente_pct > 500:

    severidade = "EMERGÊNCIA"
    falha = "CURTO-CIRCUITO"

elif corrente_pct > 130:

    severidade = "CRÍTICO"
    falha = "SOBRECARGA"

elif leitura.fator_potencia < 0.85:

    severidade = "ALERTA"
    falha = "BAIXO FP"

else:

    severidade = "NORMAL"
    falha = "NORMAL"

return DiagnosticoFalha(

    barramento=leitura.barramento,
    timestamp=leitura.timestamp,

    tipo_falha=falha,
    severidade=severidade,

    severidade_score=100 - indice_saude,

    risco_futuro=risco,

    indice_saude=indice_saude,

    causa_provavel="Análise automatizada por IA.",

    recomendacoes=[
        "Executar inspeção",

```

```
        "Monitorar parâmetros",
        "Revisar proteção"
    ],
```

```
    laudo_tecnico=f"""
    Diagnóstico inteligente detectou condição:
    {falha}
```

```

    Índice de saúde:
    {indice_saude}
```

```

    Risco futuro:
    {risco}
    """
```

```
)
```

```
# =====
# ANALYTICS
# =====
```

```
class Analytics:
```

```
    @staticmethod
    def gerar_dataframe(leituras, diagnosticos):
```

```
        dados = []
```

```
        for l, d in zip(leituras, diagnosticos):
```

```
            item = {
                "timestamp": l.timestamp,
                "barramento": l.barramento,
```

```
                "Va": l.tensao_Va,
                "Vb": l.tensao_Vb,
                "Vc": l.tensao_Vc,
```

```
                "Ia": l.corrente_Ia,
                "Ib": l.corrente_Ib,
                "Ic": l.corrente_Ic,
```

```
                "fp": l.fator_potencia,
                "temp": l.temperatura_C,
```

```
                "falha": d.tipo_falha,
                "severidade": d.severidade,
```

```
                "saude": d.indice_saude,
```



```

        "risco": d.risco_futuro
    }

    dados.append(item)

return pd.DataFrame(dados)

# =====
# DASHBOARD
# =====

class Dashboard:

    def gerar(self, df):

        fig = plt.figure(figsize=(18, 12))

        # =====
        # HEATMAP
        # =====

        plt.subplot(2, 2, 1)

        matriz = df[["Va", "Vb", "Vc", "Ia", "Ib", "Ic"]]

        sns.heatmap(
            matriz.corr(),
            annot=True,
            cmap="coolwarm"
        )

        plt.title("Correlação Elétrica")

        # =====
        # TEMPERATURA
        # =====

        plt.subplot(2, 2, 2)

        sns.barplot(
            x="barramento",
            y="temp",
            hue="severidade",
            data=df
        )

        plt.title("Temperatura")

```

```

# =====
# SAÚDE
# =====

plt.subplot(2, 2, 3)

sns.lineplot(
    x=np.arange(len(df)),
    y="saude",
    data=df,
    marker="o"
)

plt.title("Índice de Saúde")

# =====
# FP
# =====

plt.subplot(2, 2, 4)

plt.scatter(
    df["fp"],
    df["temp"],
    s=200,
    c=df["saude"],
    cmap="RdYlGn"
)

plt.colorbar(label="Saúde")

plt.xlabel("FP")
plt.ylabel("Temperatura")

plt.title("FP x Temperatura")

plt.tight_layout()

plt.savefig("dashboard_avancado.png", dpi=200)

plt.show()

# =====
# EXPORTADOR
# =====

class Exportador:

```

```

@staticmethod
def salvar(df, diagnosticos):

    df.to_csv("historico.csv", index=False)

    df.to_excel("historico.xlsx", index=False)

    with open(
        "diagnosticos.json",
        "w",
        encoding="utf-8"
    ) as f:

        json.dump(
            [asdict(d) for d in diagnosticos],
            f,
            ensure_ascii=False,
            indent=2
        )

# =====
# MONITORAMENTO CONTÍNUO
# =====

def executar():

    Logger.log("INICIANDO SISTEMA")

    simulador = SimuladorRede()

    motor = MotorDiagnostico()

    cenarios = [
        "normal",
        "sobrecarga",
        "curto",
        "desequilibrio",
        "sobretensao",
        "baixo_fp"
    ]

    leituras = []
    diagnosticos = []

    for i in range(30):

        barramento = f"BUS-{i % 6 + 1}"

```

```

cenario = random.choice(cenarios)

leitura = simulador.gerar(
    barramento,
    cenario
)

diagnostico = motor.diagnosticar(
    leitura
)

leituras.append(leitura)

diagnosticos.append(diagnostico)

Logger.log(
    f'{barramento} | '
    f'{diagnostico.tipo_falha} | '
    f'{diagnostico.severidade}'
)

time.sleep(0.1)

# =====
# DATAFRAME
# =====

df = Analytics.gerar_dataframe(
    leituras,
    diagnosticos
)

# =====
# RANKING
# =====

ranking = df.sort_values(
    by="saude"
)

print("\n===== BARRAMENTOS CRÍTICOS =====\n")

print(
    ranking[
        [
            "barramento",
            "falha",
            "saude",

```

```

        "temp"
    ]
]
)

# =====
# DASHBOARD
# =====

dash = Dashboard()

dash.gerar(df)

# =====
# EXPORTAÇÃO
# =====

Exportador.salvar(
    df,
    diagnosticos
)

Logger.log("FINALIZADO")

# =====
# EXECUTAR
# =====

executar()

```